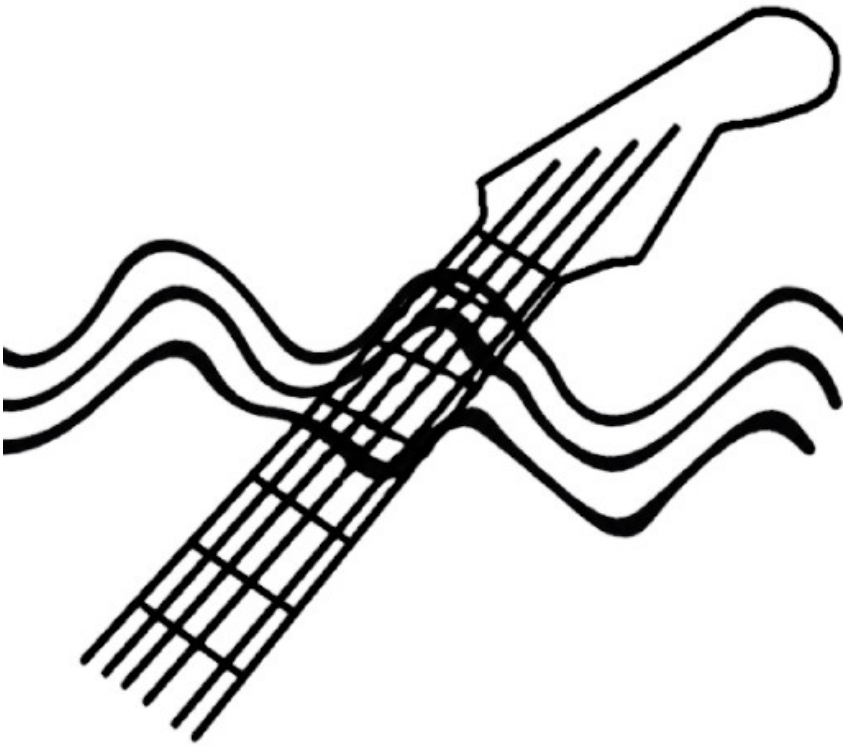
First Release - Report

Software Development Year 4 – Project



Author: Mark Lambert
Student ID: C00192497
Supervisor: Paul Barry

Submission Date: 30th October 2026

1. Introduction

The goal of the project is to build a practice tool that helps beginner guitarists change cleanly between chords. The aim is to have a mobile application that can be used with any guitar, strum some chords and get instant feedback.

This first release covers the chord and note onset detection algorithms and continuous recording within a Jupyter Notebook. This release does not implement a GUI, mobile functionality or meaningful feedback besides the chord detected and a score based on certainty of the algorithm, and a true/false result for whether each detected chord matches the prompted chord.

2. Release Targets

- Chord detection on a raw pre-recorded clip
- In-app recording functionality
- Onset detection to enable more than one chord detection per clip
- Display a bank of three chords represented as guitar tab, prompt the user to play them by initiating a recording and when stopped, detecting the chords strummed
- Report whether each detected chord matches the prompted chord

3. Research

The following applications within the same domain were researched:

- **Chordify:** Haskell, clean UI that is understandable for beginners. Chord detection on a small bank of chords for free users. Displays results dynamically, mapped onto a fretboard, as you strum chords. In-app tuner.
- **Yousician:** Python, uses **librosa,** tensorflow and pytorch libraries. Curriculum based learning, user takes set courses via video and gamification of exercises. In-app tuner.
- **Fender Play:** Video lesson focused, no active participation with app besides videos.
- **Guitariz:** Python, uses **librosa** and demucs libraries. Web based application. Uses cosine similarity.
- **Brizm.dev:** Web based application, detects chords from real time recording. Displays chords graphically and as a table with timestamps. Uses cosine similarity.
- **Chord Mini:** Python, **librosa** and chords-db for chord data. Onset detection filters strums by down vs up-strumming.

3.1. Findings

Obtaining a chroma vector by using librosa and performing cosine similarity on the vector against chord templates is the common method used by similar applications.

All applications which perform chord detection or rely on guitar input, also include an in-app tuner. This helps keep the user's attention on the application.

4. Design

4.1. Chord Detection

- `librosa.feature.chroma_stft` represents an audio file as a chroma matrix (12 pitch classes)
- Calculate the average chroma across a set of frames with `np.mean(axis=1)`
- Template chords are represented as 12 value `vectors`, to represent the pitch classes. A `1` signals a note, a `0` shows its absence
- Cosine similarity between the average chroma and each chord template is used to detect chords, the highest certainty score of the cosine similarity wins
- `detect_chord` returns the chord name, taken from the templates and the certainty score of the algorithm

4.2. Audio Recording

- `sounddevice InputStream` with a callback function allows for a continuous flow of recording that ends on termination, rather than the set recording length method when using `sounddevice.record()`
- A `time.sleep(1)` at the start of the record() function allows the user some time to get ready to strum before recording begins
- A `Queue` is used to store chunks of audio between the callback thread and main thread (producer-consumer)
- `soundfile.write()` to write result to a WAV file
- `sd.query_devices` queries the users system for the channel count on their input, rather than hard-coding the channels to 2
- Hardware tested on: AT2020 microphone through a Scarlett 2i2 for acoustic, electric guitar through Scarlett 2i2. Plus laptop microphone (Thinkpad x230)

4.3. Onset Detection

- `librosa.onset.onset_detect` returns frame indices where a spike in audio energy is marked as an onset
- Output of `onset_detect` is sensitive. When strumming a chord, more than one string is hit across a short time-frame. The plectrum hitting each string creates extra onsets, as well as general noise
- The tested '`C_to_G.wav`' clip has 16 onsets for 2 strums
- Chord detection on more than one onset:
  - Multi-dimensional array slicing the chroma between each consecutive onset using `[:, prev:cur]` where `[:,` preserves the 12 rows representing the pitch classes
- Using l`ibrosa.frames_to_time` and establishing a `buffer` (`buffer=0.5` (seconds)) enables the filtering of onsets to extract only primary strums

4.4. Chord Prompt

- A chord diagram of the 9 standard open chords is split into 1 image per chord
- Three chords are shown to the user using `random.sample()`
- Chords are displayed to the user using `Ipython.display()`
- The `record()` function mentioned in 4.2 is used to prompt the user to begin strumming the three chords
- Each chord .jpg file is named with the same name as the chord stored in the chords dictionary
- `get_chords()` in `ChordDetect.py` performs the onset detection and multi-array slicing to isolate each chord strum
- `check_chords(detected, prompted)` in `ChordDetect.py` compares each detected chord against the prompted chord, returns a list of tuples `(chord, is_correct)`

4.5. Integration

The logic and functions for chord and onset detection, recording and the bank of template chords were extracted from their notebooks and placed in `ChordDetect.py`.

4.5.1 ChordDetect.py API

`chord_templates` - Dictionary of 9 open chords (C, G, D, Dm, E, Em, Am, F, A). Each is a 12 value vector that represents the 12 pitch classes

`detect_chord(chroma_frame)` – Takes in the average chroma vector of a clip and returns the chord name and cosine similarity score

`get_chords(chords, sr)` – Takes the audio loaded by librosa and the sample rate. Detects onsets, filters them with a 0.5 second buffer and returns a list of tuples containing the chord and percent. Prints each tuple.

`check_chords(detected, prompted)` – Takes the output of `get_chords` and the list of prompted chord names. Returns a list of tuples containing the prompted chord name and a boolean value if correct or not

`record()` – Records from the default input device. Saves file to `clips/testclip.wav`. Recording ends when the notebook is interrupted. Returns the clip and samplerate

`analyse()` – Helper function used during research. Loads the clip saved by `record()` and computes the average chroma for one chord. Returns the chord and score from `detect_chord()`. Only works on a single chord recording.

5. Limitations

- The current bank of template chords is very small. For a meaningful app, need many more chords stored.
- Onset detection is still sensitive, especially when trying to play exactly three chords when prompted
- No real-time detection and feedback on transition timestamps
- `check_chords` compares chords by position. If an extra onset or a missing chord is detected, the comparisons become misaligned and no longer a usable result